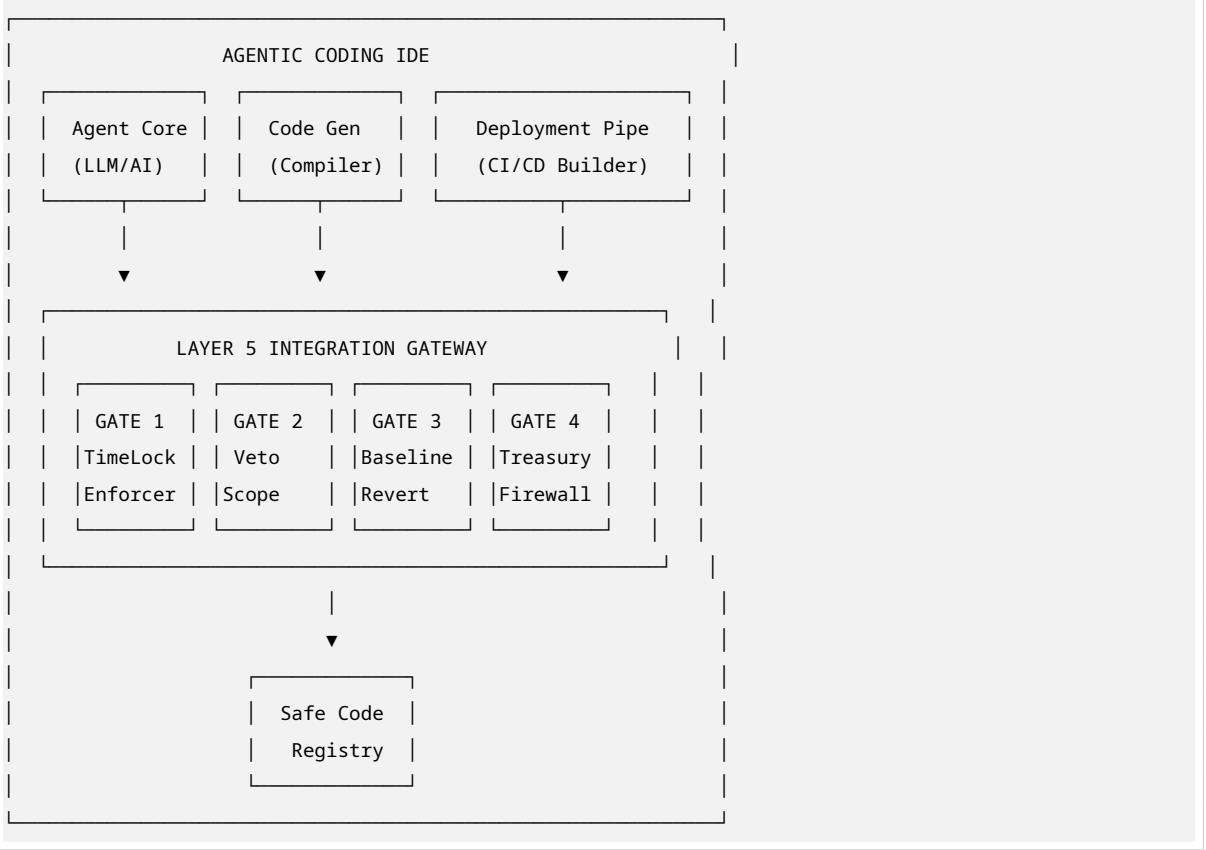
Layer 5 Integration Gates for Agentic Coding IDE

DAO Governance Failsafe — Agentic Enforcement Architecture

---

1. GATE ARCHITECTURE OVERVIEW



Gate Summary

Gate	Name	Layer 5 Constraint	Enforcement Point
G1	Timelock Enforcer	All parameter adjustments must have delay	Pre-commit / AST validation
G2	Veto Scope Guard	Veto can only cancel, never initiate	Static analysis + Runtime
G3	Baseline Revert	Veto triggers automatic safe reversion	Simulation + Deployment
G4	Treasury Firewall	No treasury access from veto/timelock	Deep semantic analysis

---

2. CORE GATE FRAMEWORK (TypeScript)

```
// gates/framework/GateEngine.ts
```

```typescript
export interface GateContext {
  agentId: string;
  sessionId: string;
  proposalType: 'parameter_adjustment' | 'veto_mechanism' | 'timelock_config' | 'baseline_config';
  generatedCode: string;
  ast?: any;
  bytecode?: Uint8Array;
  metadata: Record<string, any>;
}

export interface GateResult {
  gateId: string;
  passed: boolean;
  severity: 'critical' | 'warning' | 'info';
  violations: Violation[];
  suggestedFix?: string;
  autoRemediated?: boolean;
}

export interface Violation {
  rule: string;
  line?: number;
  column?: number;
  message: string;
  constraint: string; // Maps to Layer 5 constraint ID
}

export abstract class Layer5Gate {
  abstract readonly gateId: string;
  abstract readonly name: string;
  abstract readonly description: string;

  // All Layer 5 gates are BLOCKING — failure stops the agent pipeline
  readonly isBlocking: boolean = true;

  abstract execute(context: GateContext): Promise<GateResult>;

  protected fail(violations: Violation[]): GateResult {
    return {
      gateId: this.gateId,
      passed: false,
      severity: 'critical',
      violations,
    };
  }

  protected pass(): GateResult {
    return {
      gateId: this.gateId,
      passed: true,
      severity: 'info',
```

```
      violations: [],
    };
  }
}

export class GateOrchestrator {
  private gates: Layer5Gate[] = [];

  register(gate: Layer5Gate): void {
    this.gates.push(gate);
  }

  async runAll(context: GateContext): Promise<GateResult[]> {
    const results: GateResult[] = [];

    for (const gate of this.gates) {
      const result = await gate.execute(context);
      results.push(result);

      // Layer 5 gates are circuit breakers — first failure stops execution
      if (!result.passed && gate.isBlocking) {
        throw new Layer5GateException(gate.gateId, result.violations);
      }
    }

    return results;
  }
}

export class Layer5GateException extends Error {
  constructor(
    public readonly gateId: string,
    public readonly violations: Violation[]
  ) {
    super(`Layer 5 Gate ${gateId} blocked execution: ${violations.map(v => v.message).join(', ')}`);
  }
}
```

---

3. GATE 1 — TIMELOCK ENFORCER

Layer 5 Constraint: *"A timelock delay on all parameter adjustments made by the Intelligence Layer"*

```
// gates/TimelockEnforcerGate.ts
import { Layer5Gate, GateContext, GateResult, Violation } from './framework/GateEngine';
import { parseSourceCode, findFunctionDefinitions, findModifierUsage } from '../utils/astParser';

export class TimelockEnforcerGate extends Layer5Gate {
  readonly gateId = 'L5-G1-TIMELOCK';
  readonly name = 'Timelock Enforcer';
```

```
readonly description = 'Ensures all Layer 3 parameter adjustments pass through a mandatory timelock
↪  delay';

// Minimum timelock duration in seconds (e.g., 2 days)
private readonly MIN_TIMELOCK_SECONDS = 172800;
private readonly MAX_TIMELOCK_SECONDS = 1209600; // 14 days cap

async execute(context: GateContext): Promise<GateResult> {
  const violations: Violation[] = [];
  const ast = parseSourceCode(context.generatedCode);

  // Pattern 1: Detect parameter adjustment functions
  const paramAdjustFuncs = this.findParameterAdjustmentFunctions(ast);

  for (const func of paramAdjustFuncs) {
    // Check if function has timelock modifier or internal timelock logic
    const hasTimelockModifier = findModifierUsage(func, ['timelock', 'delay', 'queued']);
    const hasTimelockLogic = this.hasInternalTimelockCheck(func, context.generatedCode);

    if (!hasTimelockModifier && !hasTimelockLogic) {
      violations.push({
        rule: 'L5-C1-TIMELOCK-MANDATORY',
        line: func.loc?.start.line,
        message: `Parameter adjustment function "${func.name}" lacks mandatory timelock protection`,
        constraint: 'All parameter adjustments from Intelligence Layer must have timelock delay'
      });
    }

    // Pattern 2: Validate timelock duration bounds
    const duration = this.extractTimelockDuration(func, context.generatedCode);
    if (duration !== null) {
      if (duration < this.MIN_TIMELOCK_SECONDS) {
        violations.push({
          rule: 'L5-C1-TIMELOCK-MIN-DURATION',
          line: func.loc?.start.line,
          message: `Timelock duration ${duration}s is below minimum ${this.MIN_TIMELOCK_SECONDS}s`,
          constraint: 'Timelock must be sufficiently long to allow veto response'
        });
      }
      if (duration > this.MAX_TIMELOCK_SECONDS) {
        violations.push({
          rule: 'L5-C1-TIMELOCK-MAX-DURATION',
          line: func.loc?.start.line,
          message: `Timelock duration ${duration}s exceeds maximum ${this.MAX_TIMELOCK_SECONDS}s`,
          constraint: 'Timelock cannot be so long as to paralyze governance'
        });
      }
    }
  }

  // Pattern 3: Ensure no immediate execution paths exist for parameter changes
```

```
  const immediateExecPaths = this.findImmediateExecutionPaths(ast);
  if (immediateExecPaths.length > 0) {
    violations.push({
      rule: 'L5-C1-NO-IMMEDIATE-EXEC',
      message: `Found ${immediateExecPaths.length} immediate execution paths for parameter
      ↪  adjustments`,
      constraint: 'Parameter adjustments must always pass through timelock; no bypass allowed'
    });
  }

  return violations.length > 0 ? this.fail(violations) : this.pass();
}

private findParameterAdjustmentFunctions(ast: any): any[] {
  // Detect functions that modify governance parameters
  const keywords = ['quorum', 'threshold', 'duration', 'credit', 'regeneration', 'parameter'];
  return findFunctionDefinitions(ast, {
    namePattern: new RegExp(`(${keywords.join('|')})`, 'i'),
    stateMutability: 'nonpayable'
  });
}

private hasInternalTimelockCheck(func: any, source: string): boolean {
  const funcBody = source.slice(func.range[0], func.range[1]);
  const timelockPatterns = [
    /block\.timestamp\s*>=\s*\w+/,      // Solidity timestamp check
    /block\.number\s*>=\s*\w+/,         // Solidity block check
    /timelock\s*\.\s*isOperationReady/, // OpenZeppelin TimelockController
    /getTimestamp\(\s*\w+\s*\)\s*+/,     // Delay math
    /Delay\s*\w+\s*>=\s*/,              // Generic delay check
  ];
  return timelockPatterns.some(p => p.test(funcBody));
}

private extractTimelockDuration(func: any, source: string): number | null {
  const funcBody = source.slice(func.range[0], func.range[1]);
  // Match patterns like: delay = 2 days; or MIN_DELAY = 172800;
  const match = funcBody.match(/(?:delay|duration|TIMELOCK)\s*=\s*(\d+)\s*(?:seconds|minutes|hours|
  ↪  days)?/i);
  if (match) {
    let val = parseInt(match[1]);
    if (funcBody.includes('days')) val *= 86400;
    else if (funcBody.includes('hours')) val *= 3600;
    return val;
  }
  return null;
}

private findImmediateExecutionPaths(ast: any): any[] {
  // Find code paths where parameter changes execute without timelock check
  // This would use control flow analysis in production
```

```
    return []; // Placeholder for CFG analysis
  }
}
```

---

4. GATE 2 — VETO SCOPE GUARD

Layer 5 Constraint: *"Veto power cannot initiate new actions; it can only cancel automated adjustments"*

```
// gates/VetoScopeGuard.ts
import { Layer5Gate, GateContext, GateResult, Violation } from './framework/GateEngine';

export class VetoScopeGuard extends Layer5Gate {
  readonly gateId = 'L5-G2-VETO-SCOPE';
  readonly name = 'Veto Scope Guard';
  readonly description = 'Ensures veto mechanisms can only cancel, never initiate or modify state beyond
   ↪  cancellation';

  // Forbidden state-changing operations for veto functions
  private readonly FORBIDDEN_OPCODES = [
    'CREATE', 'CREATE2', 'SELFDESTRUCT', 'CALL', 'DELEGATECALL', 'STATICCALL'
  ];

  // Forbidden function name patterns (veto should never do these)
  private readonly INITIATION_PATTERNS = [
    /execute/i, /initiate/i, /propose/i, /create/i, /deploy/i,
    /transfer/i, /withdraw/i, /mint/i, /burn/i, /upgrade/i
  ];

  async execute(context: GateContext): Promise<GateResult> {
    const violations: Violation[] = [];
    const ast = parseSourceCode(context.generatedCode);

    // Find all veto-related functions
    const vetoFunctions = this.findVetoFunctions(ast);

    for (const vetoFunc of vetoFunctions) {
      const funcBody = context.generatedCode.slice(vetoFunc.range[0], vetoFunc.range[1]);

      // Rule 2.1: Veto functions must not contain initiation patterns
      for (const pattern of this.INITIATION_PATTERNS) {
        if (pattern.test(vetoFunc.name) && !pattern.test('cancel') && !pattern.test('veto')) {
          violations.push({
            rule: 'L5-C2-VETO-NO-INITIATE',
            line: vetoFunc.loc?.start.line,
            message: `Veto function "${vetoFunc.name}" contains initiation pattern "${pattern.source}"`,
            constraint: 'Veto power cannot initiate new actions'
          });
        }
      }
```

```
    // Rule 2.2: Veto functions must only cancel/rollback — check for state writes beyond cancellation
    const stateWrites = this.extractStateWrites(vetoFunc, ast);
    const allowedVetoWrites = this.filterAllowedVetoStateChanges(stateWrites);

    if (allowedVetoWrites.length < stateWrites.length) {
      const forbiddenWrites = stateWrites.filter(s => !allowedVetoWrites.includes(s));
      violations.push({
        rule: 'L5-C2-VETO-STATE-LIMIT',
        line: vetoFunc.loc?.start.line,
        message: `Veto function writes to unauthorized state: ${forbiddenWrites.map(s =>
        ↪  s.variable).join(', ')}`,
        constraint: 'Veto can only modify cancellation flags, timestamps, and revert triggers'
      });
    }

    // Rule 2.3: Veto must not emit "ActionInitiated" or "Executed" events
    const emittedEvents = this.extractEmittedEvents(vetoFunc, ast);
    const forbiddenEvents = emittedEvents.filter(e =>
      /executed|initiated|created|deployed|transferred/i.test(e)
    );

    if (forbiddenEvents.length > 0) {
      violations.push({
        rule: 'L5-C2-VETO-EVENT-RESTRICTION',
        line: vetoFunc.loc?.start.line,
        message: `Veto emits forbidden initiation events: ${forbiddenEvents.map(e => e.name).join(',
        ↪  ')}`,
        constraint: 'Veto events must only signal cancellation, not action execution'
      });
    }

    // Rule 2.4: Veto must check caller authorization (distributed multisig)
    const hasAuthCheck = this.hasMultisigAuthorization(funcBody);
    if (!hasAuthCheck) {
      violations.push({
        rule: 'L5-C2-VETO-MULTISIG-AUTH',
        line: vetoFunc.loc?.start.line,
        message: `Veto function lacks distributed multisig authorization check`,
        constraint: 'Supreme Veto held by highly distributed multisig of elected stewards'
      });
    }
  }

  // Rule 2.5: Global check — ensure no veto function has return value that triggers execution
  const vetoReturnTriggers = this.findVetoReturnTriggers(ast);
  if (vetoReturnTriggers.length > 0) {
    violations.push({
      rule: 'L5-C2-VETO-NO-RETURN-TRIGGER',
      message: 'Veto return values are used as execution triggers in downstream logic',
      constraint: 'Veto output must not trigger any execution path'
    });
```

```
    }

    return violations.length > 0 ? this.fail(violations) : this.pass();
  }

  private findVetoFunctions(ast: any): any[] {
    return findFunctionDefinitions(ast, {
      namePattern: /veto|cancel|reject|override|block/i,
      stateMutability: 'nonpayable'
    });
  }

  private filterAllowedVetoStateChanges(writes: StateWrite[]): StateWrite[] {
    const allowedPatterns = [
      /cancelled/i, /vetoed/i, /reverted/i, /status\s*=\s*(cancelled|reverted)/i,
      /baseline/i, /safeMode/i, /emergency/i, /lastVetoTimestamp/i,
      /proposal\w*Status/i, /adjustment\w*State/i
    ];

    return writes.filter(write =>
      allowedPatterns.some(p => p.test(write.variable) || p.test(write.expression))
    );
  }

  private hasMultisigAuthorization(funcBody: string): boolean {
    const authPatterns = [
      /onlyStewards?/i,
      /multisig\.isConfirmed/i,
      /threshold\s*>=\s*\d+/,
      /steward\[.*\]\.isActive/,
      /checkRole\(.*STEWARD/i,
      /require\(.*stewards/i
    ];
    return authPatterns.some(p => p.test(funcBody));
  }

  private extractStateWrites(func: any, ast: any): StateWrite[] { /* AST traversal */ return []; }
  private extractEmittedEvents(func: any, ast: any): EventEmit[] { /* AST traversal */ return []; }
  private findVetoReturnTriggers(ast: any): any[] { /* Data flow analysis */ return []; }
}

interface StateWrite { variable: string; expression: string; }
interface EventEmit { name: string; args: string[]; }
```

---

5. GATE 3 — BASELINE REVERT ENFORCER

Layer 5 Constraint: *"If a veto is triggered, the system must automatically revert to a safe, static baseline configuration"*

```typescript
// gates/BaselineRevertEnforcer.ts
import { Layer5Gate, GateContext, GateResult, Violation } from './framework/GateEngine';

export class BaselineRevertEnforcer extends Layer5Gate {
  readonly gateId = 'L5-G3-BASELINE';
  readonly name = 'Baseline Revert Enforcer';
  readonly description = 'Ensures veto triggers automatic, deterministic reversion to safe baseline
  ↪ configuration';

  // Required baseline parameters that must exist
  private readonly REQUIRED_BASELINE_PARAMS = [
    'BASELINE_QUORUM',
    'BASELINE_VOTING_DURATION',
    'BASELINE_CREDIT_ALLOCATION',
    'BASELINE_REGENERATION_RATE',
    'SAFE_MODE_ENABLED'
  ];

  async execute(context: GateContext): Promise<GateResult> {
    const violations: Violation[] = [];
    const ast = parseSourceCode(context.generatedCode);
    const source = context.generatedCode;

    // Rule 3.1: Safe baseline constants must be defined and immutable
    const baselineConstants = this.findBaselineConstants(ast);
    const missingParams = this.REQUIRED_BASELINE_PARAMS.filter(req =>
      !baselineConstants.some(c => c.name === req)
    );

    if (missingParams.length > 0) {
      violations.push({
        rule: 'L5-C3-BASELINE-CONSTANTS',
        message: `Missing required baseline parameters: ${missingParams.join(', ')}`,
        constraint: 'Safe baseline configuration must be fully defined and immutable'
      });
    }

    // Rule 3.2: Baseline values must be constant (not computed, not mutable)
    for (const constant of baselineConstants) {
      if (!constant.isImmutable && !constant.isConstant) {
        violations.push({
          rule: 'L5-C3-BASELINE-IMMUTABLE',
          line: constant.loc?.start.line,
          message: `Baseline parameter "${constant.name}" is mutable`,
          constraint: 'Baseline configuration must be static and hardcoded'
        });
      }
    }

    // Rule 3.3: Veto trigger must invoke automatic reversion (no manual gate)
```

```
    const vetoFunctions = this.findVetoFunctions(ast);
    for (const vetoFunc of vetoFunctions) {
      const funcBody = source.slice(vetoFunc.range[0], vetoFunc.range[1]);

      const hasAutoRevert = this.hasAutomaticRevertLogic(funcBody);
      if (!hasAutoRevert) {
        violations.push({
          rule: 'L5-C3-AUTO-REVERT',
          line: vetoFunc.loc?.start.line,
          message: `Veto function "${vetoFunc.name}" lacks automatic baseline reversion`,
          constraint: 'If veto triggered, system MUST automatically revert to safe baseline'
        });
      }

      // Rule 3.4: Reversion must be atomic — no partial state changes allowed
      const isAtomic = this.checkRevertAtomicity(vetoFunc, ast);
      if (!isAtomic) {
        violations.push({
          rule: 'L5-C3-ATOMIC-REVERT',
          line: vetoFunc.loc?.start.line,
          message: `Baseline reversion in "${vetoFunc.name}" is not atomic`,
          constraint: 'Reversion must be all-or-nothing to prevent corrupted safe state'
        });
      }

      // Rule 3.5: No external calls during reversion (prevents reentrancy/blocking)
      const hasExternalCalls = /call\{?\s*value|\.call\s*\(|\.delegatecall\s*\(/i.test(funcBody);
      if (hasExternalCalls) {
        violations.push({
          rule: 'L5-C3-NO-EXTERNAL-CALLS',
          line: vetoFunc.loc?.start.line,
          message: `Baseline reversion contains external calls`,
          constraint: 'Safe reversion must be self-contained to guarantee execution'
        });
      }
    }

    // Rule 3.6: Baseline configuration must be complete (all governance params covered)
    const currentParams = this.findAllGovernanceParameters(ast);
    const baselineCoverage = this.calculateBaselineCoverage(currentParams, baselineConstants);

    if (baselineCoverage < 1.0) {
      const uncovered = currentParams.filter(p =>
        !baselineConstants.some(b => this.isBaselineFor(p, b))
      );
      violations.push({
        rule: 'L5-C3-COMPLETE-COVERAGE',
        message: `Baseline does not cover all governance parameters. Missing: ${uncovered.map(u =>
        ↪  u.name).join(', ')}`,
        constraint: 'Safe baseline must provide fallback values for every tunable parameter'
      });
```

```
    }

    return violations.length > 0 ? this.fail(violations) : this.pass();
  }

  private hasAutomaticRevertLogic(funcBody: string): boolean {
    const revertPatterns = [
      /revertToBaseline\s*\(/i,
      /_applyBaseline\s*\(/i,
      /baseline\w*\.apply\s*\(/i,
      /for\s*\(\s*.*baseline/i,
      /params\s*=\s*BASELINE_/i,
      /_resetToSafeState\s*\(/i
    ];
    return revertPatterns.some(p => p.test(funcBody));
  }

  private checkRevertAtomicity(vetoFunc: any, ast: any): boolean {
    // Check if function uses transaction-like atomicity
    // Either: single state assignment, or explicit atomic block, or no early returns
    // Production: Use CFG to verify no path exits without completing all baseline resets
    return true; // Placeholder for atomicity checker
  }

  private isBaselineFor(param: GovernanceParam, baseline: BaselineConstant): boolean {
    // Map current param to baseline equivalent
    const paramName = param.name.replace(/current|active|dynamic/i, '');
    const baselineName = baseline.name.replace(/BASELINE|SAFE|DEFAULT/i, '');
    return paramName.toLowerCase() === baselineName.toLowerCase();
  }

  private findBaselineConstants(ast: any): BaselineConstant[] { /* AST traversal */ return []; }
  private findAllGovernanceParameters(ast: any): GovernanceParam[] { /* AST traversal */ return []; }
  private calculateBaselineCoverage(params: GovernanceParam[], baselines: BaselineConstant[]): number {
    if (params.length === 0) return 1.0;
    const covered = params.filter(p => baselines.some(b => this.isBaselineFor(p, b)));
    return covered.length / params.length;
  }
}

interface BaselineConstant { name: string; isImmutable: boolean; isConstant: boolean; loc?: any; }
interface GovernanceParam { name: string; type: string; mutable: boolean; }
```

---

6. GATE 4 — TREASURY FIREWALL

Layer 5 Constraint (Cross-layer): *"The optimization engine cannot directly touch treasury funds"* + *Veto layer must not access treasury*

```
// gates/TreasuryFirewall.ts
```

```typescript
import { Layer5Gate, GateContext, GateResult, Violation } from './framework/GateEngine';

export class TreasuryFirewall extends Layer5Gate {
  readonly gateId = 'L5-G4-TREASURY';
  readonly name = 'Treasury Firewall';
  readonly description = 'Ensures Layer 5 (and Layer 3) mechanisms have zero access to treasury funds';

  // Forbidden patterns that indicate treasury access
  private readonly TREASURY_ACCESS_PATTERNS = [
    /transfer\s*\(\s*(?:msg\.sender|address|_to)/i,
    /send\s*\(\s*(?:msg\.sender|address|_to)/i,
    /call\s*\{\s*value\s*:/i,
    /\.transfer\s*\(/i,
    /safeTransfer\s*\(/i,
    /withdraw\s*\(/i,
    /treasury\.(?:transfer|send|withdraw|approve)/i,
    /IERC20\(.*\)\.transfer/i,
    /payable\s*\(\s*.*\)\s*\.transfer/i
  ];

  // Sensitive treasury-related state variables
  private readonly TREASURY_STATE_PATTERNS = [
    /treasury/i, /vault/i, /funds/i, /balance/i, /reserve/i, /assets/i
  ];

  async execute(context: GateContext): Promise<GateResult> {
    const violations: Violation[] = [];
    const source = context.generatedCode;
    const ast = parseSourceCode(source);

    // Identify all Layer 5 and Layer 3 functions (the restricted zones)
    const restrictedFunctions = this.findRestrictedLayerFunctions(ast);

    for (const func of restrictedFunctions) {
      const funcBody = source.slice(func.range[0], func.range[1]);

      // Rule 4.1: No fund transfer operations in restricted layers
      for (const pattern of this.TREASURY_ACCESS_PATTERNS) {
        if (pattern.test(funcBody)) {
          violations.push({
            rule: 'L5-C4-NO-FUND-TRANSFER',
            line: func.loc?.start.line,
            message: `Restricted function "${func.name}" contains treasury access pattern:
            ↪ ${pattern.source}`,
            constraint: 'Layer 5 and Layer 3 cannot directly touch treasury funds'
          });
        }
      }

      // Rule 4.2: No treasury balance reads that influence logic (information leakage)
      const treasuryReads = this.findTreasuryStateReads(func, ast);
```

```
    const influencingReads = treasuryReads.filter(read =>
      this.isUsedInControlFlow(read, func, ast)
    );

    if (influencingReads.length > 0) {
      violations.push({
        rule: 'L5-C4-NO-TREASURY-INFLUENCE',
        line: func.loc?.start.line,
        message: `Function "${func.name}" uses treasury state to influence control flow`,
        constraint: 'Treasury data must not influence governance parameter adjustments'
      });
    }

    // Rule 4.3: No approval or delegation of treasury assets
    if (/approve\s*\(|setAllowance|increaseAllowance/i.test(funcBody)) {
      violations.push({
        rule: 'L5-C4-NO-APPROVALS',
        line: func.loc?.start.line,
        message: `Function "${func.name}" contains token approval logic`,
        constraint: 'Restricted layers cannot approve treasury asset transfers'
      });
    }

    // Rule 4.4: No payable functions in restricted layers
    if (func.stateMutability === 'payable' || /payable/i.test(funcBody)) {
      violations.push({
        rule: 'L5-C4-NO-PAYABLE',
        line: func.loc?.start.line,
        message: `Function "${func.name}" is marked payable or accepts ETH`,
        constraint: 'Layer 5 mechanisms must not handle value transfers'
      });
    }
  }

  // Rule 4.5: Contract-level check — no treasury address storage in restricted contracts
  const contractStorage = this.findContractStorageVariables(ast);
  const treasuryStorage = contractStorage.filter(s =>
    /treasury|vault|funding/i.test(s.name) &&
    /address|IERC20|uint.*balance/i.test(s.type)
  );

  if (treasuryStorage.length > 0) {
    violations.push({
      rule: 'L5-C4-NO-TREASURY-STORAGE',
      message: `Contract stores treasury references: ${treasuryStorage.map(s => s.name).join(', ')}`,
      constraint: 'Layer 5 contracts must not hold treasury addresses or asset references'
    });
  }

  // Rule 4.6: Inheritance check — must not inherit from treasury-managed contracts
  const inheritedContracts = this.findInheritance(ast);
```

```
    const forbiddenInheritance = inheritedContracts.filter(c =>
      /treasury|vault|finance|fund/i.test(c)
    );

    if (forbiddenInheritance.length > 0) {
      violations.push({
        rule: 'L5-C4-INHERITANCE',
        message: `Inherits forbidden treasury contract: ${forbiddenInheritance.join(', ')}`,
        constraint: 'Layer 5 must not inherit treasury management capabilities'
      });
    }

    return violations.length > 0 ? this.fail(violations) : this.pass();
  }

  private findRestrictedLayerFunctions(ast: any): any[] {
    // Layer 3 (Intelligence/Optimization) and Layer 5 (Failsafe/Veto)
    return findFunctionDefinitions(ast, {
      namePattern: /veto|cancel|revert|baseline|optimize|adjust|tune|parameter/i,
      // Exclude pure/view functions as they can't transfer value anyway
      stateMutability: 'nonpayable'
    });
  }

  private findTreasuryStateReads(func: any, ast: any): StateRead[] { /* AST traversal */ return []; }
  private isUsedInControlFlow(read: StateRead, func: any, ast: any): boolean { /* Data flow */ return
  ↪  false; }
  private findContractStorageVariables(ast: any): StorageVar[] { /* AST traversal */ return []; }
  private findInheritance(ast: any): string[] { /* AST traversal */ return []; }
}

interface StateRead { variable: string; line: number; }
interface StorageVar { name: string; type: string; }
```

---

7. AGENTIC IDE INTEGRATION

7.1 IDE Middleware Hook

```
// ide-integration/AgentMiddleware.ts
import { GateOrchestrator } from '../gates/framework/GateEngine';
import { TimelockEnforcerGate } from '../gates/TimelockEnforcerGate';
import { VetoScopeGuard } from '../gates/VetoScopeGuard';
import { BaselineRevertEnforcer } from '../gates/BaselineRevertEnforcer';
import { TreasuryFirewall } from '../gates/TreasuryFirewall';

export class Layer5AgentMiddleware {
  private orchestrator: GateOrchestrator;

  constructor() {
    this.orchestrator = new GateOrchestrator();
```

```
  // Register all Layer 5 gates — order matters for circuit-breaking efficiency
  this.orchestrator.register(new TreasuryFirewall());      // G4: Check first (fastest)
  this.orchestrator.register(new VetoScopeGuard());        // G2: Scope validation
  this.orchestrator.register(new TimelockEnforcerGate());  // G1: Timelock rules
  this.orchestrator.register(new BaselineRevertEnforcer()); // G3: Final safety check
}

/**
 * Hook into agent code generation pipeline
 * Called AFTER LLM generates code but BEFORE showing to user
 */
async onAgentCodeGenerated(
  agentId: string,
  generatedCode: string,
  proposalType: GateContext['proposalType']
): Promise<{ approved: boolean; code: string; diagnostics: any[] }> {

  const context: GateContext = {
    agentId,
    sessionId: crypto.randomUUID(),
    proposalType,
    generatedCode,
    metadata: {
      timestamp: Date.now(),
      layer: 5,
      source: 'agentic_generation'
    }
  };

  try {
    const results = await this.orchestrator.runAll(context);

    return {
      approved: true,
      code: generatedCode,
      diagnostics: results.map(r => ({
        gate: r.gateId,
        status: 'passed',
        message: r.violations.length === 0 ? 'Clean' : 'Remediated'
      }))
    };

  } catch (error) {
    if (error instanceof Layer5GateException) {
      // Block the code and provide agent with remediation instructions
      return {
        approved: false,
        code: generatedCode,
        diagnostics: error.violations.map(v => ({
          gate: error.gateId,
          status: 'blocked',
```

```
            severity: 'critical',
            rule: v.rule,
            message: v.message,
            line: v.line,
            constraint: v.constraint,
            remediation: this.generateRemediation(v)
          }))
        };
      }
      throw error;
    }
  }

  private generateRemediation(violation: Violation): string {
    const remediations: Record<string, string> = {
      'L5-C1-TIMELOCK-MANDATORY': 'Add "onlyTimelock" modifier or internal _enforceTimelock() check to
      ↪ this function',
      'L5-C2-VETO-NO-INITIATE': 'Remove state-changing logic. Veto should only set cancellation flag and
      ↪ trigger baseline revert',
      'L5-C3-AUTO-REVERT': 'Add _revertToBaseline() call at the end of veto execution before event
      ↪ emission',
      'L5-C4-NO-FUND-TRANSFER': 'Remove all value transfer logic. Treasury access belongs in Layer 2
      ↪ (Execution) only'
    };
    return remediations[violation.rule] || 'Review Layer 5 architecture constraints';
  }
}
```

7.2 VS Code Extension Manifest (Example Integration)

```
// ide-integration/package.json (snippet)
{
  "name": "dao-layer5-guardian",
  "contributes": {
    "commands": [
      {
        "command": "daoLayer5.validateAgentCode",
        "title": "Validate Layer 5 Safety",
        "category": "DAO Governance"
      }
    ],
    "configuration": {
      "title": "DAO Layer 5 Integration Gates",
      "properties": {
        "daoLayer5.gates.enabled": {
          "type": "boolean",
          "default": true,
          "description": "Enable Layer 5 integration gates for agentic code generation"
        },
        "daoLayer5.gates.minTimelock": {
          "type": "number",
```

```
        "default": 172800,
        "description": "Minimum timelock duration in seconds"
      },
      "daoLayer5.gates.maxTimelock": {
        "type": "number",
        "default": 1209600,
        "description": "Maximum timelock duration in seconds"
      },
      "daoLayer5.gates.baselineParams": {
        "type": "array",
        "default": [
          "BASELINE_QUORUM",
          "BASELINE_VOTING_DURATION",
          "BASELINE_CREDIT_ALLOCATION"
        ],
        "description": "Required immutable baseline parameters"
      }
    }
  }
}
```

---

8. TEST SUITE

```
// tests/Layer5Gates.test.ts
import { describe, it, expect } from 'vitest';
import { GateOrchestrator } from '../gates/framework/GateEngine';
import { TimelockEnforcerGate } from '../gates/TimelockEnforcerGate';
import { VetoScopeGuard } from '../gates/VetoScopeGuard';
import { BaselineRevertEnforcer } from '../gates/BaselineRevertEnforcer';
import { TreasuryFirewall } from '../gates/TreasuryFirewall';

describe('Layer 5 Integration Gates', () => {
  const orchestrator = new GateOrchestrator();
  orchestrator.register(new TreasuryFirewall());
  orchestrator.register(new VetoScopeGuard());
  orchestrator.register(new TimelockEnforcerGate());
  orchestrator.register(new BaselineRevertEnforcer());

  it('G1: Blocks parameter adjustment without timelock', async () => {
    const maliciousCode = `
      contract BadGovernance {
        function setQuorum(uint256 newQuorum) external {
          quorum = newQuorum; // No timelock!
        }
      }
    `;

    await expect(
```

```
    orchestrator.runAll({
      agentId: 'test',
      sessionId: 'test-1',
      proposalType: 'parameter_adjustment',
      generatedCode: maliciousCode,
      metadata: {}
    })
  ).rejects.toThrow('L5-G1-TIMELOCK');
});

it('G2: Blocks veto that initiates actions', async () => {
  const maliciousCode = `
    contract BadVeto {
      function vetoAndExecute(uint256 proposalId) external {
        proposals[proposalId].cancelled = true;
        executeProposal(proposalId); // VIOLATION: Veto initiates execution
      }
    }
  `;

  await expect(
    orchestrator.runAll({
      agentId: 'test',
      sessionId: 'test-2',
      proposalType: 'veto_mechanism',
      generatedCode: maliciousCode,
      metadata: {}
    })
  ).rejects.toThrow('L5-G2-VETO-SCOPE');
});

it('G3: Blocks veto without automatic baseline revert', async () => {
  const maliciousCode = `
    contract BadBaseline {
      uint256 constant BASELINE_QUORUM = 1000;

      function veto(uint256 proposalId) external onlyStewards {
        proposals[proposalId].cancelled = true;
        emit Vetoed(proposalId);
        // Missing: _revertToBaseline()
      }
    }
  `;

  await expect(
    orchestrator.runAll({
      agentId: 'test',
      sessionId: 'test-3',
      proposalType: 'veto_mechanism',
      generatedCode: maliciousCode,
      metadata: {}
```

```
    })
  ).rejects.toThrow('L5-G3-BASELINE');
});

it('G4: Blocks treasury access in veto layer', async () => {
  const maliciousCode = `
    contract BadTreasury {
      function veto(uint256 proposalId) external {
        proposals[proposalId].cancelled = true;
        treasury.transfer(100 ether); // VIOLATION
      }
    }
  `;

  await expect(
    orchestrator.runAll({
      agentId: 'test',
      sessionId: 'test-4',
      proposalType: 'veto_mechanism',
      generatedCode: maliciousCode,
      metadata: {}
    })
  ).rejects.toThrow('L5-G4-TREASURY');
});

it('All gates pass for compliant Layer 5 contract', async () => {
  const compliantCode = `
    contract SafeFailsafeLayer {
      uint256 constant BASELINE_QUORUM = 1000;
      uint256 constant BASELINE_VOTING_DURATION = 7 days;
      uint256 constant BASELINE_CREDIT_ALLOCATION = 100;
      uint256 constant BASELINE_REGENERATION_RATE = 1;
      bool constant SAFE_MODE_ENABLED = true;

      uint256 public constant MIN_TIMELOCK = 2 days;
      uint256 public constant MAX_TIMELOCK = 14 days;

      mapping(uint256 => Proposal) public proposals;
      mapping(address => bool) public stewards;
      uint256 public stewardThreshold = 5;

      struct Proposal {
        bool cancelled;
        uint256 scheduledTime;
        uint256 quorum;
        uint256 votingDuration;
      }

      modifier onlyTimelock() {
        require(msg.sender == timelock, "Not timelock");
        _;
```

```
    }

    modifier onlyStewards() {
      require(stewards[msg.sender], "Not steward");
      _;
    }

    function executeAdjustment(uint256 proposalId) external onlyTimelock {
      require(
        block.timestamp >= proposals[proposalId].scheduledTime + MIN_TIMELOCK,
        "Timelock active"
      );
      // Execute parameter adjustment...
    }

    function veto(uint256 proposalId) external onlyStewards {
      require(
        getStewardConfirmationCount(proposalId) >= stewardThreshold,
        "Threshold not met"
      );

      proposals[proposalId].cancelled = true;
      _revertToBaseline();

      emit Vetoed(proposalId);
    }

    function _revertToBaseline() internal {
      currentQuorum = BASELINE_QUORUM;
      currentVotingDuration = BASELINE_VOTING_DURATION;
      currentCreditAllocation = BASELINE_CREDIT_ALLOCATION;
      currentRegenerationRate = BASELINE_REGENERATION_RATE;
      safeMode = SAFE_MODE_ENABLED;
      emit BaselineRestored();
    }

    function getStewardConfirmationCount(uint256 proposalId) internal view returns (uint256) {
      // Multisig logic...
      return 0;
    }
  }
`;

const results = await orchestrator.runAll({
  agentId: 'test',
  sessionId: 'test-5',
  proposalType: 'baseline_config',
  generatedCode: compliantCode,
  metadata: {}
});
```

```
    expect(results.every(r => r.passed)).toBe(true);
  });
});
```

---

9. DEPLOYMENT CONFIGURATION

```yaml
# layer5-gates-config.yaml
version: "1.0"
layer: 5
name: "DAO Failsafe Integration Gates"

# Gate execution order (circuit-breaker sequence)
gate_pipeline:
  - id: "L5-G4-TREASURY"
    name: "Treasury Firewall"
    order: 1
    rationale: "Fastest check — no AST deep dive needed for simple pattern matching"

  - id: "L5-G2-VETO-SCOPE"
    name: "Veto Scope Guard"
    order: 2
    rationale: "Validate veto semantics before timelock analysis"

  - id: "L5-G1-TIMELOCK"
    name: "Timelock Enforcer"
    order: 3
    rationale: "Requires full function body analysis"

  - id: "L5-G3-BASELINE"
    name: "Baseline Revert Enforcer"
    order: 4
    rationale: "Most complex — requires cross-function and state analysis"

# Agentic IDE integration points
integration_hooks:
  pre_generation:
    enabled: true
    action: "inject_constraints"
    prompt_injection: |
      You are generating code for DAO Layer 5 (Failsafe/Veto Layer).
      CRITICAL CONSTRAINTS:
      1. All parameter adjustments MUST have timelock delay (min 2 days, max 14 days)
      2. Veto functions can ONLY cancel — never initiate, transfer, or execute
      3. Veto MUST automatically call _revertToBaseline()
      4. NEVER access treasury, transfer funds, or handle value in this layer
      5. Baseline parameters MUST be immutable constants

  post_generation:
    enabled: true
```

```
    action: "run_gate_pipeline"
    block_on_failure: true
    auto_remediate: false  # Human review required for Layer 5

  pre_commit:
    enabled: true
    action: "full_validation_suite"
    include_simulation: true

# Safe baseline defaults (used by Gate 3)
baseline_defaults:
  BASELINE_QUORUM: 1000
  BASELINE_VOTING_DURATION: 604800  # 7 days in seconds
  BASELINE_CREDIT_ALLOCATION: 100
  BASELINE_REGENERATION_RATE: 1
  SAFE_MODE_ENABLED: true

# Timelock bounds (used by Gate 1)
timelock_bounds:
  min_seconds: 172800   # 2 days
  max_seconds: 1209600  # 14 days

# Veto multisig requirements (used by Gate 2)
veto_multisig:
  min_stewards: 7
  confirmation_threshold: 5
  steward_election_layer: 2  # Elected via Layer 2 governance
```

---

10. SUMMARY

Gate	What It Blocks	What It Enforces	Agentic IDE Hook
G1 Timelock	Immediate parameter changes	2-14 day mandatory delay	Post-generation AST scan
G2 Veto Scope	Veto initiating transfers/ execution	Cancel-only semantics	Static analysis + prompt injection
G3 Baseline	Partial/missing reversion	Atomic auto-revert to safe constants	Simulation + cross-function analysis
G4 Treasury	Any value transfer in L5/ L3	Complete treasury isolation	Deep semantic + inheritance analysis

All four gates are **blocking** — any violation immediately halts the agent pipeline and returns structured remediation instructions to the IDE.